MEGN 301
Mechanical Integration & Design

Student Guide

Sprint-by-Sprint Tips, Warnings, and Quick References

Colorado School of Mines
Department of Mechanical Engineering

Adam Duran
Assistant Teaching Professor
PE · PMP

Spring 2026

This guide provides sprint-by-sprint tips, common pitfalls, and quick references to accompany the *MEGN 301 Master Reference Document*. Read both together.

How to use this guide. At the start of each sprint, read the corresponding section (5–10 minutes). Check the "Top Priorities" box, scan the warnings, and note any MRD chapters to review. At the end of each sprint, use the checklist to verify your deliverables are complete.

Contents

1 General Project Philosophy **3**
1.1 The Habits of Successful Teams 3
1.2 Where to Find Deeper Technical Guidance 4
1.3 The Big Picture: How Sprints Map to the Engineering V-Model 5

2 Sprint 1 – Market Analysis, SDRD, and Test Plan **6**
2.1 Market Analysis . . . 6
2.2 Patent Search . . . 7
2.3 Customer Needs (AI Persona Interviews) . . . 7
2.4 System Design Requirements Document (SDRD) . . . 8
2.5 Test Plan . . . 9
2.6 Sprint Retrospective . . . 10

3 Sprint 2 – Preliminary Design Review (PDR) **10**
3.1 Reverse Engineering . . . 10
3.2 Failure Modes and Effects Analysis (FMEA) . . . 11
3.3 PDR Report and Decision Matrix . . . 12

4 Sprint 3 – Critical Design Review (CDR) **13**
4.1 CDR Report . . . 13
4.2 Sensor and Actuator Selection . . . 15
4.3 Purchase Requests . . . 15
4.4 Manufacturing Design Review . . . 16

5 Sprint 4 – Critical Subsystem Demonstrations **17**

6 Sprint 5 – Complete System Demonstration **19**

7 Sprint 6 – External Team Testing **21**

8 Sprint 7 – Prototype Refinement and Retesting **22**

9 Final Design Oral Presentation **23**

10 Final Design Report **24**

11 General Tips for Success **26**
11.1 Time Management . . . 26
11.2 Team Dynamics . . . 27
11.3 Documentation Habits . . . 27
11.4 Prototyping and Testing . . . 27
11.5 Using Campus Resources . . . 28

12 Quick Reference: Sprint Checklist **29**

13 Quick Reference: Key Formulas and Design Rules **29**

1 General Project Philosophy

Q: What is the single most important thing to understand about this project?

You are designing and building a **non-destructive add-on** to an existing commercial product. Everything flows from this constraint: your system must interface cleanly with the parent product, add real value to the end user, and be removable without leaving a trace. Think of yourself as an aftermarket accessory company, not a product redesign firm.

Q: How should our team be thinking about "success" in this course?

Success is not just a working prototype. It is the combination of (1) a clearly defined problem, (2) a rigorous requirements document that a stranger could use to evaluate your system, (3) a traceable design process with evidence of iteration, and (4) honest reflection on what worked and what did not. A prototype that meets 80% of well-written requirements with clear documentation of why the other 20% fell short will outperform a "working" prototype with vague requirements every time.

Start a shared team engineering notebook (digital is fine: Google Docs, Notion, or OneNote all work) on Day 1. Log every decision, test result, and design change with a date and the team member responsible. This notebook will be invaluable when writing your final report and will save you hours of trying to reconstruct your design history at the end of the semester.

1.1 The Habits of Successful Teams

The difference between teams that finish strong and teams that scramble is rarely technical skill. It is process discipline. Here is what high-performing teams in this course consistently do:

Single source of truth. The BOM lives in one spreadsheet, not five. The wiring diagram is one file, not a photo on someone's phone. When two versions exist, neither is trusted. Pick one shared location (Google Drive folder with a clear naming convention) and enforce it.

15-minute standups. At the start of every work session, each member answers three questions: (1) what did I finish since last time? (2) what am I working on today? (3) what is blocking me? This takes 3 minutes per person and surfaces problems before they fester. Because the class meets only on Tuesdays and Thursdays, adapt this to a rhythm that works for your team: for example, an asynchronous check-in (a short Slack or Teams post) on Monday, Wednesday, and Friday, with a brief in-person sync on class days.

Task ownership with deadlines. "Someone should order the motors" means nobody orders the motors. Every task has one name and one date. A shared task board (Trello, Notion, even a whiteboard photo) makes ownership visible.

Pre-mortem before each review. Two days before PDR/CDR/FDR, the team imagines: "It is Thursday evening and our review just failed. What went wrong?" Then fix those things. This 20-minute exercise catches more problems than any checklist.

Design review rehearsal. Run through the presentation once, out loud, with a timer, at least 24 hours before the real review. Identify slides that take too long, claims without evidence, and questions you cannot answer. Fix them.

Documentation in real time. The team that documents as they go writes a better final report in 10 hours. The team that reconstructs from memory writes a worse report in 40 hours. Photograph every assembly step, save every oscilloscope screenshot, and commit every firmware change with a meaningful message.

1.2 Where to Find Deeper Technical Guidance

This guide tells you what to watch out for. The *MEGN 301 Master Reference Document* (MRD) tells you *how things work* in depth. It is organized in three parts: Part I follows the design-build lifecycle (Chapters 1–9: problem definition through end-of-life), Part II provides technical reference material for electromechanical subsystems (connectors, power supplies, motors, power architecture, electronics fundamentals, sensor/actuator selection, fasteners, bearings, power transmission, injection molding, PCB design, microcontrollers, and systematic debugging), and Part III covers the three formal design reviews (PDR, CDR, FDR) with deliverable checklists and evaluation criteria. The front matter includes a sprint-to-chapter mapping table, a "What Should I Be Reading Right Now?" triage guide with page numbers, the V-Model diagram, a First Week Survival Guide checklist, and a Laboratory Safety chapter. The back matter includes a comprehensive index. Use both documents together: this guide for the quick-hit tips and warnings, and the MRD for the deeper "why" and "how."

Use the MRD's navigation aids. The "What Should I Be Reading Right Now?" triage table on the first page includes page numbers for every entry. The comprehensive index at the back of the MRD lets you find any topic by keyword. Quick-reference tables in each technical chapter summarize common component selections and design values.

Understand the three questions that gate your project. The three design reviews form a narrative arc, and each asks a progressively deeper question:

1. **PDR** (Sprints 1–2): "*Should* this design work?" Is the problem clear, are the requirements sound, and is the selected concept feasible?
2. **CDR** (Sprint 3): "*Will* this design work?" Is the design complete enough to build? Every component specified, every interface defined?
3. **FDR** (Sprints 5–7 + Final): "*How well did* it work?" What are the test results? What worked, what did not, and what comes next?

If you keep these three questions in mind from the start, you will always know what your deliverables are trying to demonstrate. Part III of the Master Reference Document provides detailed deliverable checklists, evaluation criteria tables, and common failure patterns for each review.

You are maturing the same documents all semester, not starting over. Your SDRD, RTM, FMEA, BOM, VCRM, CAD models, and wiring diagrams all evolve from initial versions at PDR through detailed versions at CDR to final as-built versions at FDR. Keep every ar-

tifact under version control and update them continuously rather than treating each sprint's deliverables as isolated assignments.

Maintain a Requirements Traceability Matrix (RTM) from Day 1. The Master Reference Document introduces the RTM in Chapter 3, a simple table that links every requirement to its parent need, its verification method, and its current status. Open it at every team meeting and update it after every design decision. By the time you reach external testing in Sprint 6, a well-maintained RTM will let you immediately see which requirements are verified, which are still open, and which need attention. Teams that skip this step spend hours reconstructing traceability for the final report.

1.3 The Big Picture: How Sprints Map to the Engineering V-Model

Students often ask how the individual sprints connect to each other. The diagram below maps our sprint sequence onto the classic systems engineering V-model. The key insight is that the left side of the V (definition and design) and the right side (verification and validation) are *mirror images*: every requirement you write in Sprint 1 must have a corresponding test executed in Sprints 5–6. If you keep this traceability in mind from the start, the later sprints become dramatically smoother.

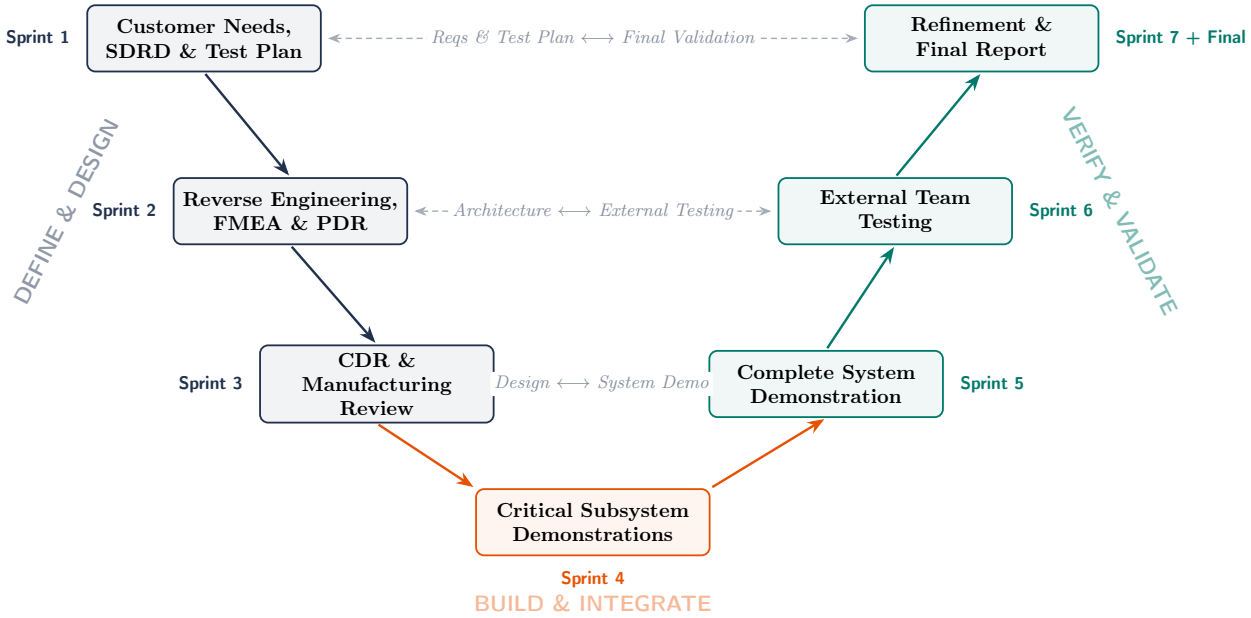

Figure 1: The MEGN 301 design project mapped onto the systems engineering V-model. Matched pairs sit at the same level: Sprint 1 requirements define the success criteria verified in the Sprint 7 final report; Sprint 2 architecture decisions are validated through Sprint 6 external testing; Sprint 3 detailed designs are verified by Sprint 5 system demonstrations. Sprint 4 sits at the vertex where design transitions to integration.

Phase 1: "Should This Design Work?"

Sprints 1–2 define the problem, write requirements, and select a concept.

2 Sprint 1 – Market Analysis, SDRD, and Test Plan

▶ Top Priorities This Sprint

1. Write testable, quantified requirements; everything downstream depends on this.
2. Draft the test plan *in parallel* with the SDRD, not after.
3. Use the market analysis to make at least one concrete design decision.

✓ Before You Start: Sprint 1 Preparation

1. Read MRD Chapters 1–3 (Problem Definition, Value Proposition, Requirements).
2. Read MRD Chapter 20 (Debugging). You will need this methodology before you touch hardware.
3. Skim this Student Guide, Sprints 1–3 sections.
4. Set up a shared team engineering notebook (Google Docs, Notion, or OneNote).
5. Map out the full semester calendar including Career Day, Presidents' Day, and Spring Break.
6. Identify the parent commercial product and purchase it (if not already provided).
7. Do the MRD Chapter 1 practice questions to calibrate your understanding.

Map out the semester calendar now. Two short weeks hit early: **Career Day (Feb. 3)** cancels your Tuesday class during Sprint 2, and **Presidents' Day Break (Feb. 17)** cancels Tuesday during the Sprint 2/3 transition. **Spring Break (Mar. 24–28)** shuts down campus, including shipping and receiving, the machine shop, and all labs. If a component fails during Sprint 4 demos (Mar. 10–12) and you need a replacement, you must order it *before* break. Build your sprint timelines around these gaps at your very first team meeting. Also note: **more than 3 unexcused absences results in automatic failure**. Being physically present but not participating (playing games, watching videos, working on other classes) counts as absent.

2.1 Market Analysis

Q: Where should I look for competing products?

Start with Amazon, specialty retailers, and Kickstarter/Indiegogo for consumer products in your space. Do not limit yourself to exact matches; look for products that solve the same underlying *problem*, even if the form factor is different. For example, if your project is the Smart Canine Nutrition Station, look at automatic pet feeders, smart pet bowls, and pet health monitoring systems.

Q: What makes a good pros/cons table?

Focus on attributes the *customer* cares about, not just specs. "Uses a 12-bit ADC" is an engineering detail; "accurately portions food within ±5 grams" is something a pet owner cares about. Good categories include price, ease of setup, reliability (check those 1-star reviews!), power source flexibility, app quality, and compatibility with existing products.

Do not treat the market analysis as a check-the-box exercise. The whole point is to identify a gap or weakness in what is currently available and use that insight to *differentiate* your design. If your market analysis does not change at least one thing about how you think about your project, you probably did not dig deep enough.

Use the Five-Element Problem Statement Framework. Before you jump to solutions, make sure you can clearly articulate: (1) **WHO** experiences the problem, (2) **WHAT** is the measurable gap between the current and desired state, (3) **WHERE** does the problem occur (physical/operational context), (4) **WHY IT MATTERS** (cost, safety, health consequences), and (5) **CONSTRAINTS** that limit the solution (budget, schedule, regulations). If your problem statement implies only one possible solution, you have jumped into the solution space too early. A good problem statement lets someone propose a fundamentally different approach. See MRD Chapter 1 (Problem Definition); use the index to find "five-element framework" and "stakeholder analysis."

2.2 Patent Search

Q: I have never read a patent before. What do I actually need to look at?

Three things: (1) the **filing/grant date** (patents expire after 20 years from filing, so anything filed before roughly 2006 is now public domain), (2) the **abstract** (a quick summary to see if it is relevant), and (3) the **claims** (usually at the very end; these define exactly what is legally protected). You do *not* need to read the entire patent specification.

Use Google Patents (`https://patents.google.com/`) and try multiple search terms. If your first search returns thousands of irrelevant results, narrow it with more specific terminology. Combining a broad term with a functional term often works well, e.g., "automatic pet feeder dispensing mechanism" rather than just "pet feeder."

2.3 Customer Needs (AI Persona Interviews)

Q: How do I get useful information out of an AI-generated persona?

Treat the AI persona like a real customer interview. Ask open-ended questions first ("Walk me through a typical morning with your dog's feeding routine") before drilling into specifics. Avoid leading questions like "Wouldn't it be great if the feeder had an app?" Instead, ask "How do you currently keep track of feeding times?" Let the persona reveal the pain points organically.

Create three personas that represent *different* user segments, not three variations of the same person. For example, for the Medication Assistant: a caregiver managing multiple patients, an independent elderly person living alone, and a busy parent managing a child's medication. Different personas surface different needs.

GenAI Policy Boundary: The AI persona interview assignment *requires* you to use Gemini Pro (via your Mines email). However, this is the **only** approved GenAI use in the course. You may *not* use GenAI to write your SDRD requirements, generate your test plan, compose your FMEA, write your retrospectives, or produce your reports. GenAI is the *subject* of this exercise, not your co-author. Any unapproved use of GenAI technologies is an honor code violation.

2.4 System Design Requirements Document (SDRD)

Q: What is the difference between an objective and a requirement?

An **objective** is qualitative: it describes *what* the system should do or be. A **requirement** is quantitative: it specifies a measurable threshold that proves the objective is met. For example, Objective: "The system shall dispense food accurately." Requirement: "The system shall dispense the user-specified amount of food within ±10% by weight."

Q: How do I know if a requirement is well-written?

Apply the "stranger test": could someone who has never seen your project read the requirement and know *exactly* what to measure, how to measure it, and what constitutes a pass or fail? If the answer is no, the requirement needs work. Every requirement must be clear, quantifiable, and testable. To illustrate, here is the same requirement at three levels of quality:

Level	Requirement
Bad	"The system shall be durable."
Better	"The system shall survive a drop."
Best	"The system shall maintain full mechanical and electrical functionality after a 1-meter drop onto a concrete surface, verified by executing the full test suite immediately after the drop."

The "Bad" version is untestable. "Better" adds a physical event but leaves out the height, surface, and acceptance criteria. "Best" specifies everything a tester needs: drop height, surface material, and a concrete pass/fail procedure. Aim for "Best" on every requirement.

Vague requirements like "the system shall be easy to use" or "the system shall be reliable" are the number one source of problems later in the semester. They are impossible to test, which means you cannot demonstrate that your prototype meets them. Rewrite them: "A first-time user shall be able to complete the setup process in under 5 minutes without external assistance" is testable.

Write your requirements and your test plan *at the same time*. If you cannot immediately envision how you would test a requirement, it is either too vague or unmeasurable. This back-and-forth process naturally produces better requirements.

Every technical choice must trace to a requirement. When you select a motor, connector, or PCB trace width in Part II of the Master Reference Document, look for the Requirements Mapping box at the end of each chapter. It shows exactly how to translate a datasheet parameter into a formal shall-statement. If you cannot write a requirement for a component choice, you have not justified the selection.

Prioritize requirements using MoSCoW. Not all requirements are equally critical. Classify each as: **Must** (system fails without it, becomes a SHALL requirement), **Should** (important but system works without it, becomes a SHOULD goal), **Could** (nice-to-have, pursue if budget allows), or **Won't** (explicitly excluded from this version). This prevents scope creep and helps your team focus on the requirements that matter most when time gets tight. See MRD Chapter 1 (search the index for "MoSCoW prioritization").

2.5 Test Plan

Q: How detailed does the test plan need to be?

Detailed enough that a team who has *never seen your project* can execute every test. Remember, in Sprint 6 an external team will be running your test plan. If they have to guess what you meant, you will get ambiguous results. Specify equipment needed, step-by-step procedures, pass/fail criteria, and expected duration for each test.

Q: Does every single SDRD requirement need a corresponding test?

Yes. If a requirement exists in your SDRD but has no test, then you have no way to verify that requirement is met. Either write a test for it or reconsider whether it should be in the SDRD. There should be a one-to-one (or many-to-one) mapping from tests to requirements.

Know the four verification methods (TAID). Not every requirement is verified by running a physical test. The four methods are: **Test** (measure performance under controlled conditions), **Analysis** (verify by calculation, simulation, or modeling), **Inspection** (verify by visual or dimensional examination), and **Demonstration** (verify by operating the system through its use cases). Assign a verification method to each requirement when you write it. Most of your requirements will use Test or Demonstration, but some (e.g., material compliance, dimensional tolerances) may be verified by Inspection or Analysis. See MRD Chapter 5 (Verification, Validation & Test Plans); the TAID methods figure and test procedure template are in the first three pages of that chapter.

2.6 Sprint Retrospective

Q: What is the point of the retrospective?

The retrospective is not busywork; it is a structured opportunity to improve your team's process. Be honest about what went well, what did not, and what you will change. The best teams use retrospectives to catch problems early (e.g., unbalanced workloads, communication gaps, unclear task ownership) before they become crises.

3 Sprint 2 – Preliminary Design Review (PDR)

▶ Top Priorities This Sprint

1. Thoroughly reverse engineer the COTS product, especially its electrical interfaces.
2. Complete an honest FMEA; do not downplay severity to look "clean."
3. Weight your decision matrix criteria *before* scoring alternatives.

✓ Before You Start: Sprint 2 Preparation

1. Read MRD Chapters 4 (Concept Generation) and 21 (PDR deliverables).
2. Review your Sprint 1 requirements. Are they all testable and quantified?
3. Obtain the COTS product and identify all interfaces (mechanical, electrical, data).
4. Prepare functional decomposition before selecting any components.
5. Gather materials for the morphological chart (at least 3 options per function).

Start with functional decomposition before jumping to components. Before picking sensors, microcontrollers, or actuators, list the *functions* your system must perform using verb-noun pairs: Sense Temperature, Dispense Food, Filter Air, etc. Then break each function into sub-functions. This separates *what* the system must do from *how* it does it, and it directly feeds your trade study. A morphological chart (listing functions as rows and 3–5 implementation options as columns) is the most powerful way to systematically generate meaningfully different design concepts. See MRD Chapters 3–4 (Requirements & Concept Generation); follow the teal GreenShield boxes for worked examples.

3.1 Reverse Engineering

Q: What am I actually looking for when I reverse engineer the COTS product?

You are trying to answer: "What constraints does this existing product impose on my add-on design?" Look for mounting points, electrical interfaces (voltage, current, connector types), physical dimensions and tolerances, material properties, and any features that could either help or hinder your integration. Document everything with photos and measurements.

Measure twice (or three times). Tolerances on consumer products can be surprisingly loose. If you are designing a mechanical interface, build in adjustability or compliance rather than relying on a single measurement from one unit.

Explicitly document the **connectors and power draw** of the COTS product during reverse engineering. One of the most common integration failures occurs when an add-on system pulls more current than the parent product's power supply can handle, or when the team orders a mating connector with the wrong pitch (e.g., 2.0 mm vs. 2.54 mm JST headers). Record connector type, pin count, pitch, voltage, and rated/measured current draw. For any stainless steel fluid fittings, apply nickel anti-seize compound before assembly to prevent thread galling (a cold-weld that permanently destroys both parts). This information is critical for your CDR and will prevent expensive, time-consuming mistakes in Sprints 3–4.

3.2 Failure Modes and Effects Analysis (FMEA)

Start your FMEA now, not at FDR. The Master Reference Document positions FMEA as a design-phase tool (Chapters 2–3), not a post-mortem. Performing FMEA during architecture enables you to design out critical failure modes before ordering parts. Waiting until the final report turns FMEA into an autopsy.

Q: How do I identify failure modes I have not thought of yet?

Systematically walk through every component and every interface in your system and ask: "What happens if this breaks, jams, shorts, overheats, receives the wrong input, or loses power?" Also consider misuse scenarios: what if the user does something unexpected? The USCSB ExxonMobil video assigned in this sprint is an excellent example of how overlooked failure modes cascade into catastrophic outcomes.

Do not assign uniformly low severity or occurrence ratings to make your FMEA look "clean." An FMEA with no high-risk items is almost certainly an FMEA that was not done thoroughly. Instructors will see through this immediately. The value of the FMEA is in *finding* the scary stuff so you can address it in your design.

Score the FMEA using three independent scales. Each failure mode gets rated on Severity (how bad if it happens, 1–10), Occurrence (how likely, 1–10), and Detection (how likely the failure goes unnoticed, 1–10). Multiply them to get the Risk Priority Number: $\text{RPN} = S \times O \times D$. Failure modes with $\text{RPN} \geq 100$ generally need corrective action in your design. A high-severity, low-occurrence, hard-to-detect failure (e.g., temperature sensor reading high so frost is not detected ($S = 9, O = 3, D = 5, \text{RPN} = 135$), demands more attention than an obvious, low-impact failure. Prioritize your design resources based on RPN, not gut feeling.

Think about all six types of interfaces during your FMEA. Failures cluster at interfaces, not inside components. The six interface categories are: electrical (voltage, protocol, grounding), mechanical (mounting, tolerances, alignment), thermal (heat paths, contact resistance, component overheating), software/data (APIs, data format, timing), fluid (fittings, pressure, seal compatibility), and human (display visibility, control layout, error prevention). Walk through each interface in your system and ask what happens when each one fails.

3.3 PDR Report and Decision Matrix

Q: How many design alternatives should we consider in the trade study?

At minimum, you should have three meaningfully different concepts. "Meaningfully different" means different architectures or approaches, not minor variations of the same idea (e.g., "same design but in blue" does not count). The decision matrix should use weighted criteria that trace back to your customer needs and SDRD objectives.

When building your decision matrix, weight the criteria *before* scoring the alternatives. This prevents the common bias of adjusting weights after the fact to justify the concept you already wanted to build.

Run a sensitivity analysis on your decision matrix. After scoring, vary each weight by ± 0.05 and check whether the winning concept changes. For instance, if Concept A beats Concept B by only 0.05 points, and shifting the "cost" weight by 10% makes Concept B the winner, your decision is not robust; you need better data on that criterion or should prototype both options. A margin of less than 5% between the top two concepts demands additional investigation, not a hasty commitment. Also ensure every candidate is a legitimate contender: including a deliberately weak "straw man" option to make your preferred concept win is a common pitfall that instructors spot immediately.

Q: How do I know if my PDR package is ready?

Apply the **PDR Litmus Test**: "If a different engineering team read only our PDR package, could they independently arrive at the same concept selection and understand why it is the best choice?" If the answer is yes, your PDR is defensible. If the answer requires "well, they would also need to know that. . . ," that missing information must be added to the package. Remember, PDR answers the question "*Should* this design work?" Your deliverables must demonstrate that the problem is well-defined, the requirements are sound, and the selected concept is feasible.

Phase 2: "Will This Design Work?"

Sprints 3–4 freeze the design, procure parts, and demonstrate critical subsystems.

4 Sprint 3 – Critical Design Review (CDR)

▶ Top Priorities This Sprint

1. Submit purchase requests as early as possible; do not wait for the CDR deadline.
2. Design polymer parts for injection molding, even though you are prototyping with 3D printing.
3. Get every manufactured part reviewed by the appropriate resource *before* fabrication.

✓ Before You Start: Sprint 3 Preparation

1. Read MRD Chapter 22 (CDR deliverables) and Chapter 12 (Power Architecture).
2. Read MRD technical chapters for your project's subsystems (Ch 10–19 as relevant).
3. Build a power budget spreadsheet listing every component's voltage, current, and peak draw.
4. Identify Plan B components for every critical part in your BOM.
5. Check lead times and place early orders for long-lead or sole-source components.
6. Start a wiring diagram and pin mapping table before writing firmware.

4.1 CDR Report

Q: What does "everything someone would need to build your prototype" actually mean?

Imagine handing your CDR to a competent engineering team that has never met you. Could they purchase the correct components, fabricate every part, assemble the system, and load the software? If not, something is missing. This means complete CAD drawings with dimensions and tolerances, a full bill of materials with part numbers and vendors, wiring diagrams, software architecture descriptions, and assembly instructions.

Build a power budget spreadsheet before anything else. List every component with its voltage rail, continuous current draw, and peak current draw. Sum the columns and add a 25% margin. This single spreadsheet will tell you whether your power supply is adequate, what regulator stages you need, and where your thermal hotspots will be. Motor inrush is the number-one power supply problem: a motor rated at 2 A running may draw 15 A at startup. Size your supply for inrush, not steady state. See MRD Chapter 12 (Power Architecture & Circuit Safety); the power budget spreadsheet example and protection components quick-reference table are in that chapter.

Example power budget (3 rows to get you started):

Component	Rail	Cont.	Peak	Notes
ESP32 DevKit (Wi-Fi active)	5 V USB	240 mA	350 mA	TX bursts
DS18B20 temperature sensor	3.3 V	1.5 mA	1.5 mA	Parasitic mode: 0 mA
12 V DC gearmotor + DRV8871	12 V	2.0 A	15 A	Stall = 15 A; size fuse here
3.3 V rail total		242 mA	352 mA	Need: 440 mA supply (25% margin)
12 V rail total		2.0 A	15 A	Need: 2.5 A continuous + bulk cap for inrush

Copy this format into a spreadsheet, add every component in your BOM, and sum each rail. If the peak column exceeds your supply rating, you have a problem to solve *before* you order parts.

Separate your power and signal grounds. Poor grounding is the most common cause of hard-to-diagnose intermittent failures in student projects: noisy sensor readings, MCU resets when a motor starts, dropped serial communication. Run motor power on dedicated ground wires back to the supply; never share ground paths between motors and sensitive electronics. Use star-point grounding: each subsystem gets its own ground conductor, and they all meet at one common point near the power supply. See MRD Chapter 12; search the index for "star-point grounding" for the complete three-domain strategy.

Identify a "Plan B" component for every critical item. For any single-source or sole-vendor component in your BoM, record a backup part number and vendor in a notes column. If your specialty sensor, MOSFET, or pump goes out of stock the week before CDR, you need to order the backup *immediately*, not start a new search. This is especially important for projects with specialty components like the *Sous Vide Supreme* (heating elements) and *Sol-Mate Purification Pal* (solar charge controllers), where lead times can exceed two weeks.

4.2 Sensor and Actuator Selection

Q: How do I choose the right sensor for my project?

Use the six-criteria framework from Chapter 14 of the MRD: (1) What physical quantity? (2) What range and accuracy does your requirement specify? (3) What bandwidth (fastest change)? (4) What output type does your MCU accept? (5) What is the operating environment? (6) Budget? Match these to candidate sensors, then prefer the one with the simplest signal conditioning. A $5 digital I2C sensor that plugs directly into the ESP32 may be a better choice than a $2 analog sensor that needs a bridge, amplifier, and anti-alias filter.

Every actuator needs a driver circuit. No motor, solenoid, or heater connects directly to a GPIO pin. Before you order an actuator, verify you also have the driver (H-bridge for DC motors, ESC for BLDC, stepper driver for steppers, MOSFET + flyback diode for solenoids, SSR for AC heaters). If the driver is not in your BoM, add it now. See Chapter 11 (Motors & Pumps) and Chapter 12 (Power Architecture) for driver selection.

Check sensor output compatibility before ordering. Three things must match: (1) output voltage range fits your ADC (a 10 V sensor on a 3.3 V ESP32 will saturate and may damage the ADC), (2) output impedance is compatible (>100:1 ratio with ADC input impedance), and (3) response time is fast enough ($\tau_{\text{sensor}} < f_{\text{signal}}/5$). Chapter 14 has the full selection framework and specification tables.

4.3 Purchase Requests

Q: When should we submit our purchase request?

As early as possible. Do not wait until the CDR deadline. Shipping delays are real, and if a component arrives late, your build schedule slips. Identify long-lead-time items (specialty sensors, custom PCBs, specific actuators) early and flag them to your instructor. Have a backup vendor or component in mind for critical items.

Only order from the approved vendor list. Unapproved vendors require a review and may be denied, which costs you time. Check the list before you fall in love with a component that is only available from a vendor not on the list.

The Spring Break Shipping Trap. Spring Break begins March 24. Campus shipping/receiving, the machine shop, and all lab facilities are **closed** that week. If a component fails during your Sprint 4 subsystem demo (around Mar. 10–12) and you need a replacement for the full system demo, you must order it *immediately*, not after break. Plan your procurement so all critical components arrive before March 21. The same logic applies to 3D prints and machined parts: submit them early enough to allow for at least one iteration before campus shuts down.

4.4 Manufacturing Design Review

Q: Why do polymer components need to be designed for injection molding?

Even though your prototype will likely be 3D printed, designing for injection molding teaches you to think about manufacturability at scale. This means uniform wall thickness, draft angles, avoiding undercuts, and proper rib and boss design. The white paper provided on Canvas covers these principles in detail. This is a skill that will serve you in every future design role.

Q: Which 3D printing process should I choose?

Use **filament (FDM) printing** for larger structural components where surface finish is less critical. Use **resin (SLA) printing** for small, detailed parts that require tighter tolerances or smoother surfaces. Refer to Appendix A of the project document for resin printer settings. Get your designs reviewed early; the machine shop assistants, Prof. Blood, and the Master TAs are all available to help.

Use heat-set inserts or captive nuts for any structural fastening into 3D-printed parts. Students frequently try to tap threads directly into FDM prints or rely on superglue for structural joints. Both approaches fail under load and vibration, often catastrophically during a live demo. Brass heat-set inserts (installed with a soldering iron) create strong, reusable M3/M4 threads in printed parts for pennies each. Design the appropriately sized pilot hole into your CAD model from the start. This is a small investment that prevents mid-demo disasters.

Complete your Order of Operations sheet *before* walking into the machine shop. This is required for machined parts and is also just good practice; it forces you to think through the fabrication sequence, which often reveals design issues before you waste material.

Choose your resin strategically to save budget. The campus resin printers support two materials at very different price points: Elegoo ABS-Like at \$15/kg and Siraya Tech Blu Nylon-Like at \$38/kg (see Appendix A of the project document for settings). Use the cheaper ABS-like resin for basic structural enclosures, brackets, and cosmetic parts. Reserve the Nylon-like resin only for parts requiring high toughness, wear resistance, or repeated flexure (gears, snap-fit clips, or living hinges). A single strategic resin choice can save \$20–40 of your project budget.

Document every electrical connection and every software flow *now*, not later. Your CDR should include a wiring diagram showing every component, wire color, gauge, connector part number, and pinout, plus a programming flowchart showing your main loop, state machine transitions, sensor reads, actuator control logic, and error handling. These diagrams are the electrical and software equivalents of your CAD drawings. Without them, no one (including your future self at 10 PM during Sprint 5) can debug your system. See MRD Chapter 6 (Subsystem Integration) for interface documentation standards and the N^2 diagram.

Decide early: dev boards or custom PCB? Use development boards (Arduino, ESP32 DevKit, sensor breakouts) when your circuit is straightforward, form factor is not critical, the timeline is tight, or the design is still evolving. Use a custom PCB when a specific enclosure size is required, noise-sensitive analog signals demand a ground plane, or you need clean cable routing and high reliability. A well-wired dev board that meets requirements is superior to a custom PCB that delays the project. If you do go custom, the total timeline from design start to working board is 4–6 weeks (design, order from JLCPCB/PCBWay, ship, solder, debug), so your design must be frozen at CDR. See MRD Chapter 18 (PCB Design Fundamentals); the JLCPCB specs quick-reference table and component placement strategy are especially useful.

ESP32 GPIO gotchas that will cost you a day of debugging. Three critical rules: (1) **GPIO 6–11** are off-limits (internal flash), (2) **GPIO 34–39** are input-only with no pull-ups, and (3) **ADC2 is disabled when Wi-Fi is active** (use ADC1 channels only). Safe GPIOs for general use: 4, 13, 14, 16–19, 21–23, 25–27, 32, 33. **Print the complete GPIO pin map table from MRD Chapter 19 (Microcontrollers & the ESP32) and tape it to your workbench.** The MRD chapter also covers boot-sensitive pins, power modes, battery sizing, communication protocols, and firmware best practices.

Q: How do I know if my CDR package is ready?

Apply the **CDR Litmus Test**: "If I handed this CDR package to a competent technician who has never seen this project, could they build the prototype from these documents alone?" If the BOM is complete, the drawings are dimensioned, the wiring diagram is unambiguous, and the assembly sequence is clear, your CDR is ready. If the answer requires "well, they would also need to know that…," that missing information must be added. CDR answers "*Will* this design work?" It is the point of no return where you commit resources to procurement and fabrication.

Save a "CDR baseline" snapshot of all your documents. After CDR, you will compare your as-designed BOM, budget, and drawings against the as-built reality at FDR. Create a frozen copy of your CDR package (BOM, CAD, wiring diagrams, firmware version, budget) before you start building. This makes the as-designed vs. as-built comparison in your final report straightforward instead of a painful reconstruction exercise.

5 Sprint 4 – Critical Subsystem Demonstrations

▶ Top Priorities This Sprint

1. Get your demo proposal approved *before* this sprint begins.
2. Choose the two highest-risk subsystems, not the easiest ones.
3. Record a narrated video and verify the link works in incognito mode.

Q: How do I choose which two subsystems to demonstrate?

Pick the two subsystems that represent the highest technical risk in your design. These are the parts you are least certain will work, or the parts that, if they fail, would require the biggest redesign. The purpose of this sprint is to retire risk early. Demonstrating something you are already confident about wastes this opportunity.

Q: What if my demo does not fully meet the proposed goals?

That is okay; this is expected to happen sometimes, and it is why the demo report asks you to document what modifications are needed. The important thing is to (1) clearly state what worked and what did not, (2) provide specific performance data (not just "it kind of worked"), and (3) outline a concrete plan to address the gaps before the full system demo.

Get your demo proposal approved by your instructor *before* Sprint 4 begins. If you wait, you risk demonstrating something your instructor considers too easy or not aligned with your project's critical path, and you will have to redo it.

When you record your demonstration video, narrate what you are doing and what the expected vs. actual results are. A silent video of a blinking LED tells the viewer nothing. Also, verify your video link is accessible by opening it in an incognito/private browser window. Unviewable links may receive a zero.

Never "big-bang" integrate. Assembling every component at once and powering on is the fastest route to an undebuggable system. Instead, use the bottom-up debugging methodology from Chapter 20 of the Master Reference Document: verify each component individually, then add **one new element at a time** and test after each addition. When something breaks, the last thing you added (or its interface) is the prime suspect. For most electromechanical projects, a good integration sequence is: (1) power supply first (everything depends on it), (2) controller (the central hub), (3) sensors (simplest interfaces), (4) actuators (highest power, most failure modes), (5) communication (can be tested last since it is not needed for core function). MRD Chapter 20 covers the debugging pyramid (Figure 1 of that chapter), signal walk technique, and the five-step protocol flowchart.

✓ Debugging Checklist (Sprints 4–5)

When something does not work, follow this sequence before calling for help:

1. **Reproduce**: can you make the failure happen reliably?
2. **Visual inspection**: solder bridges, loose wires, wrong pin, reversed component?
3. **Power check**: are all voltage rails correct under load? (Multimeter, not assumption.)
4. **Continuity check**: is VCC shorted to GND? Are signal paths continuous end-to-end?
5. **Isolate**: disconnect subsystems one at a time. Does removing one fix the problem?
6. **Signal walk**: inject known input, measure at each stage with oscilloscope. First deviation localizes fault.

7. **One change at a time**: change one thing, test, record. Never change multiple variables simultaneously.
8. **Log everything**: date, symptom, measurements, hypothesis, fix, result.

See MRD Chapter 20 for the complete five-step protocol and the debugging pyramid.

(1) Visual inspection under $\geq$10$\times$ magnification (loupe or USB microscope) for solder bridges on fine-pitch ICs, cold joints, tombstoned 0603 passives, and wrong-orientation parts. A teammate must co-sign the inspection before power is applied. (2) Continuity check with a multimeter; confirm no shorts between VCC and GND. (3) Current-limited power: set your bench supply to 50 mA limit and apply voltage. (4) Measure every power rail with a multimeter. (5) Smoke test: increase current to operating level, monitor for 60 seconds. (6) Load firmware and test basic I/O. Skipping this sequence and plugging a freshly soldered board straight into your system is how you destroy both the board and the system in one step. See MRD Chapter 18; the "10$\times$ Visual Inspection Before Power-On" box has the full protocol.

Run multiple trials, not just one. A subsystem that works once under ideal bench conditions may fail intermittently under real operating conditions. For each demonstration goal, run at least 3 trials (5 is better). Report the mean, standard deviation, and range of your measurements. For example, for a temperature sensor demo, report: "Mean error = 0.32 °C, $\sigma = 0.04$ °C, range 0.26–0.41 °C, $n = 5$ trials at each of three test points," not just "it read the right temperature." Test at boundary conditions, not just room temperature on a benchtop. A result of "PASS with 36% margin" is far more convincing, and more useful, than a bare "PASS."

Phase 3: "How Well Did It Work?"

Sprints 5–7 execute V&V, collect evidence, and deliver the final package.

6 Sprint 5 – Complete System Demonstration

▶ Top Priorities This Sprint

1. Demonstrate end-to-end functionality from the user's perspective.
2. Update the BoM with actual costs and a labor estimate.
3. Finalize the test plan so it is ready for the external team; this is your last chance to fix ambiguities.

Q: What should I focus on for the in-class system demo?

Show that your system works *end-to-end* as a user would experience it. Start with the user powering on the system, walk through the primary use case, and demonstrate key requirements being met. If something is not working, be upfront about it. Instructors value honesty and a clear understanding of what went wrong far more than a polished demo that hides problems.

Q: My design has changed a lot since the CDR. Is that a problem?

Not at all; design iteration is expected and encouraged. What matters is that you *document* the changes: what changed, when it changed, and why. Part A of the Complete System Demo Report specifically asks for this. A well-documented design evolution shows engineering maturity.

When debugging integration failures, follow the five-step protocol from Chapter 20 of the MRD. (1) **Reproduce**: make the failure happen reliably and note the exact conditions. A failure you cannot reproduce is a failure you cannot fix. (2) **Isolate**: disconnect modules one at a time until the failure stops; the last module disconnected is the prime suspect. (3) **Characterize**: measure signals at the interface with an oscilloscope or multimeter and compare to your ICD specifications. (4) **Root-cause**: determine whether the problem is hardware, software, or an interface specification error. (5) **Fix and regression-test**: implement the fix, verify it resolves the original failure, then *re-run all previously passed tests* to confirm the fix did not break anything else. **Change ONE variable at a time.** If you swap firmware and a component simultaneously, you will not know which change fixed it, and the other may have introduced a new latent defect.

Use this sprint to finalize your test plan for the external team. Read through every test procedure as if you have never seen the project before. If any step is ambiguous, fix it now. You will not be in the room when the external team runs your tests.

✓ Test Plan Review Checklist (Before External Testing)

1. Every requirement has a test procedure with a unique ID
2. Each test has explicit, quantified pass/fail criteria (not "works correctly")
3. Setup instructions include specific equipment, settings, and connections
4. Preconditions are stated (warm-up time, calibration, initial state)
5. Steps are numbered and unambiguous; a stranger can follow them
6. Safety warnings appear before the step that creates the hazard
7. Data recording format is specified (table, units, sig figs)
8. Expected results are stated so the tester can judge pass/fail on the spot
9. VCRM is complete with all requirements mapped to test procedures

Do not forget to update your Bill of Materials (Part B) with actual costs, including any components you ended up not using or had to reorder. Also include a labor estimate; this is easy to overlook but is required.

Build a Verification Cross-Reference Matrix (VCRM) before the system demo. The VCRM is a simple table linking each requirement to its verification method, test procedure ID, and current status (Open, Verified, Failed, Waived). By Sprint 5, you should be able to fill in most of this table. It becomes the single artifact that tells the complete project story at your final review, and it makes writing the final report dramatically easier. See MRD Chapter 5; the VCRM evolution figure shows how the matrix matures from PDR through FDR.

When a test fails, investigate root causes in order. Three possible sources, checked in this sequence: (1) Is the *test* wrong? (procedure error, miscalibrated equipment, incorrect setup), (2) Is the *implementation* wrong? (the most common cause (a component, connection, or code issue)), (3) Is the *requirement* wrong? (overly conservative or based on flawed assumptions). Only consider changing a requirement after ruling out the first two, and any requirement change must be formally documented with rationale. Never silently adjust a requirement to make a failed test "pass."

7 Sprint 6 – External Team Testing

▶ Top Priorities This Sprint

1. Hand off a clean, well-labeled prototype with a complete test plan.
2. Observe the "Silent Observer" rule: do not coach the testing team.
3. When *you* are the testing team, be thorough and professional.

Q: What happens during external team testing?

Another team will receive your prototype and your test plan, and they will execute your tests and document the results. You will *not* be present to explain anything. This is the ultimate test of your documentation quality. Every ambiguity, missing step, or unclear pass/fail criterion in your test plan will surface here.

Prepare a handoff package, not just a prototype. Before handing your system to the external team, prepare: (1) a one-page quick-start guide with numbered steps and photos showing how to power on, configure, and operate the system, (2) all required test equipment, cables, and fixtures laid out and labeled, (3) clear photographs of the correct system configuration for each test, (4) any safety precautions highlighted prominently, and (5) your test plan reviewed one final time as if you have never seen the project. External testing typically reveals three categories of problems: procedure ambiguities ("connect the sensor" without specifying which port), missing preconditions (the system needs 5 minutes of warm-up but the procedure says nothing about it), and actual system defects that your team never encountered because they operate the system differently.

✓ External Testing Handoff Package Checklist

1. One-page quick-start guide with numbered steps and annotated photos
2. Complete test plan with pass/fail criteria for each test (no ambiguity)
3. All required test equipment, cables, adapters, and fixtures, labeled
4. Safety precautions sheet (highlighted, not buried in the manual)
5. Photographs of correct system configuration for each test
6. Power supply/charger (fully charged batteries if applicable)
7. Spare consumables (e.g., extra sensor probes, calibration samples)
8. Data recording forms or templates for the testing team to fill in
9. Contact information for emergencies only (not for coaching)

The "Silent Observer" Rule: During external testing, the designing team must remain **completely silent**. Do not coach, correct, hint, or demonstrate. If the testing team is struggling with a step, that is data; it means your documentation needs improvement. The moment someone says "Oh, you just need to press it like this," you have invalidated the test of your documentation. Resist the urge. Let the test plan speak for itself. This is one of the most valuable learning experiences in the entire course.

Q: What if the external team breaks something or does a test wrong?

This is a real-world scenario: products must survive being used by people who did not design them. If a test is performed incorrectly, it likely means your test procedure was unclear. Use this feedback constructively. If something physically breaks, document it and address it in Sprint 7.

When you are the *testing* team for another group, be thorough and fair. Your primary goal is not to make the other team look bad; it is to provide honest, constructive feedback that helps them improve. Document everything that is ambiguous or unclear in the test plan; that feedback is a gift to the design team. The quality of your external testing report reflects your own engineering professionalism, and your teammates will return the favor. This is also an opportunity to see how another team approached a different design problem; you will learn things that make you a better engineer.

8 Sprint 7 – Prototype Refinement and Retesting

▶ Top Priorities This Sprint

1. Fix every issue identified in external testing, prioritized by impact on core requirements.
2. Re-test every fix before the final presentation.
3. Begin drafting the final report and presentation; do not treat this sprint as a break.

Q: There are no formal submissions for Sprint 7. What should I be doing?

Three things: (1) address every issue identified during external team testing, (2) prepare your Final Design Oral Presentation, and (3) begin drafting your Final Design Report. This sprint is your buffer; use it wisely. Teams that treat Sprint 7 as a vacation week invariably produce weaker final deliverables.

Prioritize fixes by impact. If external testing revealed that a core requirement failed, fix that first. Cosmetic issues and nice-to-haves come last. Make sure any fixes are retested before the final presentation. After each retest, update your VCRM with the new results; this ensures the final VCRM is complete and current when you submit it as part of the FDR package.

Regression-test after every fix. Any change to your system (firmware update, component swap, wiring modification) may break something that previously worked. After implementing each fix, re-run all tests affected by that change. A fix to the motor driver firmware might shift the EMI spectrum and corrupt your sensor ADC readings. Without regression testing, you may fix one problem and silently introduce three others. Use your VCRM to track which requirements need re-verification after each change.

As you refine your prototype and prepare final deliverables, explicitly trace your results back to the course learning outcomes. Can you articulate which design decisions demonstrate your ability to define requirements, analyze trade-offs, manage risk, and verify performance? This traceability matters for course assessment, and it is exactly the kind of reflective practice that will distinguish you in Capstone, senior design, and industry.

9 Final Design Oral Presentation

Q: How should we structure a 10-minute presentation?

A recommended breakdown: introduction and problem statement (1 min), design requirements overview (1.5 min), design evolution from concept to final prototype (3 min), performance results and testing summary (2.5 min), future improvements and lessons learned (2 min). Practice with a timer. Ten minutes goes faster than you think, and you want to leave time for the 5-minute Q&A.

Do not try to cover everything. Your final report is where the details live. The presentation should tell a compelling *story*: what problem did you solve, how did your design evolve, did it work, and what did you learn? Slides crammed with text and tiny font sizes are a sign that you are trying to say too much.

Anticipate questions. Common ones include: "Why did you choose this approach over the alternatives?" "What was your biggest design challenge?" "If you had another semester, what would you change?" "How much would this cost at production volume?" Prepare concise answers to these in advance.

10 Final Design Report

Q: How much of the final report can come from earlier sprint submissions?

Much of it *should* come from earlier sprints; that is by design. However, "copy and paste from Sprint 3" is not sufficient. You are expected to **revise and update** every section to reflect the final state of your project and address any issues identified in earlier sprint feedback. Treat this as a polished, standalone document that tells the complete story.

Q: What goes in the appendix vs. the main body?

The main body should contain your narrative, key results, and high-level design information. The appendix is for detailed supporting material: full CAD drawings, complete BoM, raw test data, detailed calculations, wiring diagrams, and software documentation. A good rule of thumb: if removing it would break the narrative flow of the main body, it belongs in the main body. If it is supporting evidence or reference material, it belongs in the appendix.

From Evidence to Narrative. The final report is not a compilation of sprint deliverables stapled together. It is a *story* with a structure:

1. **Introduction** sets the scene: the problem, the stakeholders, why it matters.
2. **Design Evolution** is the rising action: concept selection, key decisions, what changed and why.
3. **Results** is the climax: VCRM verdicts, test data, system performance vs. requirements.
4. **Lessons Learned** is the resolution: what worked, what did not, what comes next.

Each sprint's deliverables become *supporting evidence* in the appendix, not the main text. In the body, summarize: "The power subsystem was redesigned between CDR and FDR to address a motor inrush brown-out (see Appendix C for the full change log). The revised design eliminated the 3.3 V rail sag measured during Sprint 4 testing." Do not paste in the raw Sprint 4 report. A reader should be able to understand the full project story from the main body alone, then consult the appendices for proof.

Q: What should go in the "Future Work" and "Hazards Discussion" sections?

For **Future Work**: think in terms of lifecycle and sustainability. The Master Reference Document includes a formal Design for Disassembly requirement: "The system shall be fully disassembled into its base material categories using only standard hand tools in under 5 minutes." This is testable at FDR by demonstration. If your design uses permanent adhesives where fasteners would work, document why and propose a DfD-compliant redesign for the next version. What refinements would bring the prototype to a production-ready product? Consider serviceability (can a technician access and replace the most failure-prone components?), maintenance planning (what are the wear items and their replacement intervals?), and a spare parts list with vendor part numbers. Thinking about Operations & Maintenance is not an afterthought; it accounts for 50–70% of a product's lifecycle cost. For the **Hazards Discussion**: revisit your FMEA with updated RPN scores. Discuss the highest-RPN failure modes you identified, what design changes you made to mitigate them, and whether those changes actually reduced the risk. Also address any new hazards discovered during prototyping and testing that were not in the original FMEA. See MRD Chapter 7 (Operations & Maintenance) for serviceability principles and the lifecycle cost figure, and Chapter 9 (End of Life) for disposal planning.

Q: How honest should the "Lessons Learned" section be?

Very. This section is not about making yourself look good; it is about demonstrating that you grew as engineers. The best lessons learned sections candidly discuss what went wrong, why it went wrong, and what the team would do differently. Instructors can tell the difference between generic filler and genuine reflection. Compare these two examples:

Quality	Example
Generic	"We learned that communication is important and that we should start earlier."
Specific	"We underestimated the integration complexity between the sensor subsystem and the control logic, which cost us two weeks. Starting in Sprint 4, we adopted a shared engineering notebook with daily entries, which prevented the miscommunications that had plagued Sprints 1–3. Next time, we would prototype the critical interface first."

The generic version could have been written without doing the project. The specific version demonstrates real engineering growth and gives actionable advice to anyone who reads it.

Each section must start on a new page (page break) so that it aligns correctly in Gradescope. Forgetting this formatting requirement can cause grading issues. Double-check before submitting.

For the Team Contribution table, have an honest team discussion before filling it out. Agree on rankings together rather than each person filling it out independently and discovering disagreements at submission time.

Q: How do I know if my FDR package is complete?

Apply the **FDR Litmus Test**. Ask six questions: Could a new engineering team who has never seen this project (1) understand what the system does and why, (2) operate it using only your Quick-Start Guide, (3) identify what works and what does not from the VCRM, (4) reproduce any test result from your test procedures, (5) replace a failed component using the maintenance plan and spare parts list, and (6) continue improving the design using your lessons learned and future work recommendations? Each "no" represents a gap in your documentation. FDR answers "*How well did* it work?" It demands honest, evidence-based accounting, not a sales pitch.

Prepare an as-designed vs. as-built comparison. Your final report should include side-by-side comparisons of your CDR (as-designed) BOM vs. actual (as-built) BOM, and your estimated budget vs. actual expenditures. Every component substitution, addition, or deletion should be documented with a rationale. If you saved a CDR baseline snapshot during Sprint 3 (see the CDR tips above), this comparison is easy. If you did not, this will be a painful reconstruction.

Include O&M materials in the appendix. Your final report should include three operations and maintenance deliverables: (1) a one-page **Quick-Start Guide** with numbered steps and photos that enables your instructor to set up and operate the system in 10 minutes, (2) a **maintenance plan** identifying wear items, replacement intervals, and a spare parts list with part numbers and vendors, and (3) a **troubleshooting guide** organized as a symptom-based decision tree for the most common failure modes you discovered. These are not busywork; they demonstrate that you thought about the system beyond your own workbench.

Trace your results back to the course learning outcomes. The final report is your opportunity to demonstrate growth across all 10 CLOs listed in the syllabus. As you write each section, ask: which CLO does this evidence support? For example, your design evolution narrative demonstrates CLO 1 (apply the engineering design process) and CLO 4 (build a functional prototype with rapid iterations); your testing summary demonstrates CLO 7 (design experiments) and CLO 9 (collect and analyze data). Making these connections, even informally, strengthens your report and helps your instructor see the full scope of your learning.

11 General Tips for Success

11.1 Time Management

- **Front-load the hard parts.** Start with whatever you are most uncertain about. The biggest regret teams have at the end of the semester is "we should have started building/coding/testing earlier."

- **Respect lead times.** Components take time to ship. The machine shop has a queue. Resin prints can fail and need reprinting. Build buffer into every timeline.
- **Set internal deadlines 48 hours before the real ones.** This gives you time to integrate, review, and fix issues before submission. Last-minute scrambles produce sloppy work and missed details.

11.2 Team Dynamics

- **Define roles and ownership early.** Every major subsystem and every deliverable should have one person who is ultimately responsible, even if others contribute.
- **Communicate proactively.** If you are stuck, behind schedule, or unsure about something, tell your team *now*, not the night before a deadline. No one can help you with a problem they do not know about.
- **Use the retrospectives honestly.** They exist specifically to surface and address team process issues. If something is not working, the retrospective is the time and place to address it constructively.

11.3 Documentation Habits

- **Document as you go.** Taking a photo of your prototype before and after every modification takes seconds and saves hours when writing the final report.
- **Version control your CAD and code.** Use Git (even GitHub Desktop if the command line is intimidating) for all firmware and software. For CAD files, use a consistent naming convention in a shared Google Drive folder: `Enclosure_V3-2_2026-04-15.SLDPRT` is infinitely better than `Final_Final_reallyFinal.SLDPRT`. Whatever system you choose, start it on Day 1, not the night before CDR.
- **Keep your BoM current.** Update it every time you order, swap, or discard a component. Reconstructing a BoM from memory at the end of the semester is painful and error-prone.
- **Document *why*, not just *what*.** The most perishable artifact in any engineering project is the rationale for design decisions. Record why you chose this sensor over that one, why you changed the mounting approach in Week 8, and what known limitations or workarounds exist. Your final report's "Lessons Learned" and "Design Evolution" sections will practically write themselves if you have been recording rationale all along.
- **Maintain a running Lessons Learned log.** As a team, spend 5 minutes at the end of every sprint adding to a shared document: "What went well? What went wrong? What will we change?" This single log will make writing your final "Lessons Learned" section a 15-minute task instead of a painful, hours-long reconstruction.

11.4 Prototyping and Testing

- **Test early and test often.** Do not wait until Sprint 5 to see if your electronics and mechanical systems work together. Bench-test individual components as soon as they arrive.

- **Have a "Plan B" component.** For any single-source critical component, know what your fallback is if it is out of stock, damaged on arrival, or does not perform as specified.
- **Design for debugging.** Include test points in your circuits, print housings that can be opened, and write code with serial debug output. Your future self debugging at 10 PM will appreciate it.
- **Use keyed connectors everywhere.** If your design has multiple similar-looking electrical connections, specify keyed connectors that can only be inserted one way. Incorrect mating is one of the most common causes of electrical failure on first power-up in student projects, and it destroys components.
- **Never design your own AC-DC converter.** Always use a pre-certified (UL/CE listed) wall adapter. These are fully enclosed, safety-rated, and eliminate all mains-voltage hazards from your design. If your project involves lithium batteries, a Battery Management System (BMS) is mandatory for overcharge, deep-discharge, and short-circuit protection. Never charge lithium cells unattended; a LiPo fire releases toxic gas and can close a lab. Always charge in a fireproof LiPo safety bag. See MRD Chapter 12 (Power Architecture); search the index for "lithium battery safety" and "BMS."
- **If you design a custom PCB, print it at 1:1 scale and place real components on the printout before ordering.** This single step catches 80% of footprint errors, the most expensive PCB mistake, because a wrong footprint means a scrapped board. Also: place a 100 nF decoupling capacitor within 5 mm of every IC power pin, and fill your bottom layer with a ground copper pour. These are not optional.
- **Use state machines and `millis()`-based timing in your firmware.** Avoid `delay()` calls, which block the CPU and make your system unresponsive. Structure your code around states (IDLE, READING, TRANSMITTING, SLEEPING) and use non-blocking timing. Enable the watchdog timer for crash recovery. Add a heartbeat LED as a system health indicator; if the LED stops blinking, something has hung.

11.5 Using Campus Resources

- **Bulk wire, heat shrink, and perf boards** are available on campus for free or at minimal cost (see Appendix B of the project document). Wire is available in BBW160 from TAs during office hours: 18 AWG stranded at \$0.19/ft, 24 AWG stranded at \$0.08/ft. Solderable perf boards are stocked in CTLM123 in several sizes. Include these per-unit costs in your BoM for accurate accounting, but do not waste budget ordering them from vendors.
- **Get manufacturing reviews early.** The machine shop assistants, Prof. Blood, and Master TAs can catch design-for-manufacturing issues before you waste material and time. This review is required, but even if it were not, you should do it.
- **Office hours exist for a reason.** Your instructor and TAs have seen dozens of these projects. Five minutes of their input can save you days of going down the wrong path.
- **Use the Master Reference Document.** The *Master Reference Document* is a comprehensive technical reference organized in three parts: Part I covers the V-model design-build lifecycle with worked GreenShield examples, Part II provides detailed electromechanical design guidance

including connectors, power supplies, motors, power architecture, fasteners, bearings, power transmission, injection molding, PCB design, and microcontroller selection, and Part III walks through PDR, CDR, and FDR deliverables with evaluation rubrics. Every Part II chapter ends with a Requirements Mapping box showing how to translate technical specs into testable shall-statements. Consult the Quick Reference Tables appendix for consolidated design values.

12 Quick Reference: Sprint Checklist

The following table provides a high-level overview of deliverables by sprint. Always refer to the official project document and Canvas for exact submission requirements and deadlines.

Sprint	Focus	Key Deliverables
1	Market Analysis, SDRD, Test Plan	Market analysis, patent search, AI persona interviews, SDRD, test plan, retrospective
2	Preliminary Design Review	Reverse engineering analysis, FMEA, PDR report with decision matrix, retrospective
3	Critical Design Review	CDR report, manufacturing design review, purchase order request, retrospective
4	Critical Subsystem Demos	Demo proposal (before sprint), demo videos, demo report, retrospective
5	Complete System Demo	System demo report (state, BoM, updated docs, revised test plan), in-class demo, retrospective
6	External Team Testing	External testing report (performed by another team)
7	Refinement and Retesting	No formal submissions; fix issues, prepare final presentation and report
–	Final Presentation	10-minute presentation + 5-minute Q&A
–	Final Report	Comprehensive report combining all sprint work (revised and updated)

13 Quick Reference: Key Formulas and Design Rules

✓ Formulas You Will Use

- **Motor torque**: $\tau = K_t \times I$ Back-EMF: $V_e = K_e \times \omega$
- **Gear ratio**: $GR = N_2/N_1 = \omega_1/\omega_2 = \tau_2/\tau_1$
- **Lead screw linear speed**: $v = \text{lead} \times \text{RPM}/60$
- **Bolt torque**: $T = K \cdot F \cdot d$ ($K = 0.20$ dry, 0.15 lubed)
- **Bearing life**: $L_{10} = (C/P)^p \times 10^6/(60n)$ hours
- **Battery runtime**: $t = C_{\text{mAh}}/I_{\text{mA}}$ hours (derate 20%)
- **RPN**: Severity × Occurrence × Detection (scale 1–10 each)
- **Availability**: $A = \text{MTBF}/(\text{MTBF} + \text{MTTR})$

✓ Design Rules to Memorize

- **Current margin**: size wires and traces at ≥125% of max continuous current
- **Torque margin**: select a motor whose continuous rated torque is ≥1.5× the calculated load torque
- **Decoupling**: 100 nF ceramic ≤5 mm from every IC power pin
- **Star-point grounding**: separate signal, power, and safety ground conductors
- **Flyback diodes**: mandatory on every inductive load (motor, relay, solenoid)
- **Thread engagement**: steel 1.5d, aluminum 2.0d, plastic 2.5d
- **ESP32 safe GPIOs**: 4, 13, 14, 16–19, 21–23, 25–27, 32, 33
- **ESP32 off-limits**: GPIO 6–11 (flash); 34–39 (input only); ADC2 (Wi-Fi conflict)
- **One change at a time**: during debugging, never change two variables simultaneously

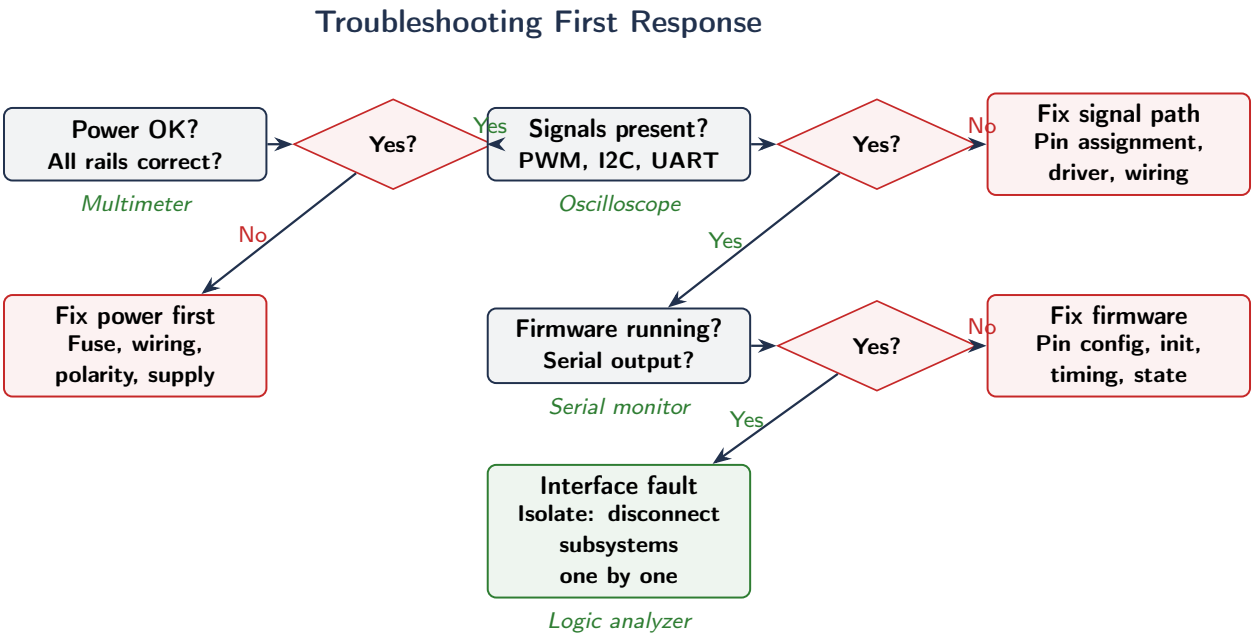

Figure 2: Troubleshooting first response. Follow left to right, top to bottom. Fix power first (most common cause), then check signals, then firmware. If all three check out, the fault is at an interface; isolate by disconnecting subsystems one at a time. See MRD Chapter 20 for the full five-step debugging protocol.

Good luck this semester. Design with intention, document with discipline, and learn from every iteration.
